UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

LEMBAR KERJA MAHASISWA

Pertemuan 3 · Enkapsulasi, Property, dan Objek Nilai yang Beku

Nama	Muhammad Izaz Albatani	**NIM**	2555201011
Kelas	Kelas A	**Tanggal**	15-10-2026
Tautan repositori	https://github.com/izazalbatani01/PBO-UP-2026.git		

A. KUIS AWAL — MENJEMBATANI DARI PERTEMUAN 2 (10 MENIT)

1. Ketika Anda menulis alat.biaya_sewa(3), berapa argumen yang sebenarnya diterima method itu, dan apa saja?

Ketika menulis alat.biaya_sewa(3), method tersebut sebenarnya menerima 2 argumen, yaitu:

1. self — objek AlatTani yang sedang digunakan, dikirim secara otomatis oleh Python.
2. jumlah_hari — nilai 3 yang diberikan saat method dipanggil.

2. Mengapa parameter berdefault wajib diletakkan di belakang?

Parameter berdefault wajib diletakkan di belakang parameter biasa karena Python harus dapat menentukan argumen mana yang diberikan saat method dipanggil.

Jika parameter berdefault diletakkan di depan parameter wajib, Python akan menghasilkan SyntaxError karena urutan parameternya tidak valid.

3. Pada Pertemuan 2 kita menulis alat._tarif_harian = -5000 dan Python menerimanya. Apa akibatnya bagi nota sewa?

Jika kita menulis alat._tarif_harian = -5000, Python tetap menerimanya karena atribut tersebut belum memiliki perlindungan validasi secara langsung.
Akibatnya, ketika menghitung biaya sewa, hasilnya bisa menjadi negatif. Misalnya, menyewa alat selama 3 hari dengan tarif -5000 akan menghasilkan biaya -15000.
Hal ini membuat nota sewa menjadi tidak masuk akal dan dapat menyebabkan kesalahan dalam pencatatan pembayaran.

4. Apa maksud prinsip “beri perintah, jangan bertanya” pada objek?

Prinsip “beri perintah, jangan bertanya” (Tell, Don't Ask) berarti kita sebaiknya meminta objek melakukan suatu tugas melalui method yang dimilikinya, daripada mengambil data internal objek lalu mengolahnya sendiri dari luar.

B. PENGAMATAN PRAKTIKUM

B.1 Konstruktor yang melewati penjaga (Langkah 4)

Pertanyaan	Jawaban Anda
Berapa tarif yang berhasil masuk lewat konstruktor?	**-5000** berhasil masuk lewat konstruktor versi bocor, karena konstruktor menulis langsung ke atribut internal self._tarif_harian sehingga melewati setter.
Apakah setter menolak ketika dipanggil langsung? Buktinya?	Ya. Saat atribut tarif_harian diisi -5000 secara langsung, setter menolak dan memunculkan ValueError (tarif harus lebih dari nol).
Satu karakter apa yang membedakan versi bocor dan versi benar?	Satu karakter garis bawah (_): versi bocor menulis self._tarif_harian = tarif_harian (melewati setter), versi benar menulis self.tarif_harian = tarif_harian (lewat setter).
Menurut Anda, mengapa kesalahan seperti ini sulit ketahuan?	Karena program tetap berjalan tanpa error meski datanya tidak valid; kesalahan baru terlihat saat data dipakai menghitung biaya atau mencetak nota sewa.

B.2 Garis bawah ganda dibongkar (Langkah 8)

Salin ketiga baris keluarannya, lalu jawab pertanyaan di bawahnya.

```
{'_Rekening__saldo': 100000}
False
100000
```

1. Nama apa yang sebenarnya dipakai Python untuk menyimpan __saldo, dan bagaimana Anda mengetahuinya?

Python mengubah __saldo lewat *name mangling* menjadi _NamaKelas__saldo. Nama sebenarnya bisa diketahui dengan memeriksa vars(objek) atau objek.__dict__, yang menampilkan nama atribut hasil perubahan itu.

2. Kalau garis bawah tidak benar-benar mengunci, apa sesungguhnya yang menjaga kode Anda?

Yang menjaga kode adalah enkapsulasi lewat property, setter, validasi, dan aturan pemakaian objek. Garis bawah ganda hanya mengubah nama atribut, bukan pengunci mutlak.

B.3 Bukti ketiga gerbang hijau (Langkah 10)

Gerbang	Keluaran

`periksa_pbo.py --profil p03`	LOLOS
`mypy`	Success: no issues found in 7 source files
`pytest`	31 passed

C. AKTIVITAS KELOMPOK: MENENTUKAN TINGKAT PENJAGAAN

Anggota kelompok: _____ aidil,izaz

Data	Tingkat Penjagaan	Alasan
kode keramba	Baca-saja	Identitas keramba tidak boleh berubah sembarangan setelah objek dibuat.
luas (m²)	Baca-saja	Luas adalah karakteristik keramba yang konsistensinya perlu dipertahankan.
jumlah bibit	Baca-tulis dengan validasi	Jumlah bibit bisa berubah, tetapi nilainya tidak boleh negatif.
lokasi	Objek nilai beku	Lokasi terdiri atas beberapa data yang menjadi satu kesatuan dan tidak boleh diubah sembarangan.
nama pemilik	Baca-tulis dengan validasi	Pemilik bisa berganti, tetapi nama tidak boleh kosong.
tanggal tebar	Baca-saja	Tanggal tebar adalah catatan kejadian yang perlu dipertahankan.

C.1 Data yang justru tidak perlu dijaga

Sebutkan satu, beserta alasannya. Butir ini bernilai paling tinggi.

Teks pembuka/judul layar (mis. pesan sambutan program) tidak perlu dijaga dengan property dan validasi ketat. Nilainya hanya untuk tampilan dan perubahannya tidak memengaruhi aturan bisnis maupun konsistensi objek, sehingga penjagaan ketat hanya menambah kode tanpa manfaat. Penjagaan difokuskan pada data yang menentukan validitas dan perilaku sistem.

D. REFLEKSI PENUTUP

1. Bandingkan kelas AlatTani Anda pekan lalu dan pekan ini. Kodenya menjadi lebih panjang. Menurut Anda apa yang didapat sebagai gantinya?

Menurut saya, meskipun kode kelas AlatTani pekan ini menjadi lebih panjang karena adanya property, getter, setter, dan validasi, ada banyak manfaat yang diperoleh.

Atribut seperti kode dan nama menjadi tidak bisa diubah sembarangan, sedangkan tarif_harian memiliki validasi agar nilainya selalu positif dan tidak melebihi batas yang ditentukan.

2. Sebutkan satu hal yang belum Anda pahami, dan pertanyaan yang akan Anda ajukan pada Pertemuan 4.

Satu hal yang belum saya pahami sepenuhnya adalah bagaimana cara menerapkan enkapsulasi pada program yang memiliki banyak kelas agar setiap objek tetap dapat berinteraksi tanpa harus mengakses atribut internal objek lain.

Pertanyaan untuk Pertemuan 4:

“Bagaimana cara menghubungkan beberapa kelas dalam Python agar dapat saling berinteraksi dengan baik tanpa melanggar prinsip enkapsulasi?”

Paraf Asisten / Dosen	Tanda Tangan Mahasiswa
……	……